



National University



of Computer & Emerging Sciences, Islamabad

**Department of Software Engineering
(Formal Methods)**

Project Description

Semester: Fall 26 (FM)

Course Instructor: Nigar Azhar Butt

Section: All

Instructions

- Read the instructions very carefully.
- Late submissions will not be accepted
- **Plagiarism may result in zero marks in the whole assignments category (all assignments) regardless of the percentage plagiarized.**
- There will be no credit if the given requirements are changed.
- This is a group project (upto 4 members).

Submission Guidelines

1. **Software Application:** A complete, functional tool for XML input, logic translation, MWP identification, and model visualization. A zip file with the naming convention `<section>_<group_id>_software.zip` e.g. (A_G01_software.zip)
2. **User Guide:** Instructions on using the software. A pdf file with the naming convention `<section>_<group_id>_user_guide.pdf` e.g. (A_G01_user_guide.pdf)
3. **Demo Video:** (maximum 5 minutes with voice-over) A walkthrough demonstrating key features of the tool. A mp4 file with the naming convention `<section>_<group_id>_demo.mp4` e.g. (A_G01_demo.pdf)

Project Overview

This project consists of two parts:

Part 1: Feature Model Analysis Tool

Part 2: Feature-to-Code Impact Analysis

Sample XML feature models have been provided. During evaluation, a new unseen XML may be given.

Part 1: Feature Model Analysis Tool

Feature models are a core component in software product line engineering, representing the hierarchical structure and constraints of system features. In this project, you will develop a comprehensive software tool that allows users to input a feature model in XML format, translate it into propositional logic, and visualize it interactively, verifying feature selections against the constraints defined in the model.

1. XML Parsing and Input Handling:

- The software should accept an XML file as input, depicting the feature model. The XML must represent features, their hierarchical relationships (mandatory, optional, XOR, OR), and cross-tree constraints (e.g., requires, excludes). The XML file has been provided as an example. During demo we may give you a new one.

2. Translation to Propositional Logic:

- The tool should parse the input XML and convert the feature model into a propositional logic formula. The translation rules must be documented clearly. For example:
 - **Root:** B (if B is root)
 - **Mandatory Feature:** $B \rightarrow A$ (If a feature B has a mandatory child A).
 - **Optional Feature:** $B \leftrightarrow (A \vee \neg A)$ (If A is optional under B).
 - **OR Group:** $(A \vee B)$ (Any (combination of) features in an OR group can be selected).
 - **XOR Group:** $(A \wedge \neg B) \vee (B \wedge \neg A)$ (Only one feature in an XOR group can be selected).
 - **Cross-Tree Constraints**

3. Minimum Working Product (MWP) Identification:

- The software should compute a list representing all "Minimum Working Products (MWP)" — a minimal valid configuration of features satisfying all

mandatory constraints while minimizing optional features and ensuring cross tree constraints.

4. Feature Model Visualization and Interaction:

- The tool must visualize the feature model as a checkbox tree, similar to software installation wizards, with the following interactive features:
 - **Hierarchical Display:** Mandatory and optional features should be visually distinguished.
 - **XOR Group Handling:**
 - If a feature in an XOR group is selected, all other features in the group should be automatically disabled.
 - **Dynamic Constraint Checking:**
 - If a feature is selected, the tool should dynamically validate the selection based on the model constraints, adjusting related features accordingly.
 - If a mandatory feature is deselected, all features requiring it should also be deselected.

5. Feature Selection Verification:

- After users have selected features, the tool should verify whether the chosen configuration is valid according to the feature model constraints.
- **Feasibility Check:**
 - Display a message indicating whether the selected product configuration is feasible or not.
 - If the configuration is invalid, provide a detailed reason, such as unmet mandatory features or violated cross-tree constraints.

6. Cross-Tree Constraints Handling:

- Cross-tree constraints (e.g., "A requires B" or "A excludes B") can be provided in either English language or propositional logic format.
 - **English to Propositional Logic Translation:**
 - If a cross-tree constraint is entered in English (e.g., "Feature A requires Feature B"), prompt the user to translate it into propositional logic (e.g., $A \rightarrow B$).
 - If a translation already exists, ask the user for confirmation to use it.
 - If no translation is provided, prompt the user to input one, which can be left empty if the user decides not to provide it.

Part 2: Feature-to-Code Impact Analysis

Feature models represent the intended design of a system, while codebases reflect the actual implementation. In this part of the project, you will extend your tool to link feature models with a real software codebase and analyze how feature selections impact implementation artifacts. You will also identify inconsistencies between feature-level constraints and code-level dependencies.

1. Codebase Input and Handling:

The software should accept a small codebase as input, either as:

- A local project folder, OR
- A Git repository link

The selected codebase must:

- Contain 10–25 files
- Be written in any programming language

The system should display basic information about the codebase, such as:

- Folder structure
- List of files/modules

2. Feature-to-Code Mapping:

The tool should allow users to map features from the feature model to implementation artifacts (files/modules). The mapping should be represented clearly (e.g., table or UI view).

Requirements:

- At least 5 features must be mapped to code artifacts
- Each feature must map to at least one file

The mapping must be:

- Justified (not arbitrary)
- Based on functionality of the code

3. Code Dependency Extraction:

The tool should identify dependencies between files/modules in the codebase.

Dependencies may include:

- Import/include statements
- Module usage relationships

Examples:

payment.py → auth.py

checkout.js → payment.js

Requirements:

- At least 5 dependencies must be identified

Note:

- Full static analysis is NOT required
- A semi-manual or lightweight approach is sufficient

4. Dependency Representation:

The system should represent extracted dependencies in a clear form, such as:

- A list of dependencies, OR
- A simple dependency graph

The representation should show:

- Source file
- Dependent file

5. Feature-Level Dependency Derivation:

Using:

- Feature-to-code mapping
- File-level dependencies

The tool should infer feature-level relationships. If a file mapped to Feature A depends on a file mapped to Feature B, then Feature A depends on Feature B. The inferred dependencies should be displayed clearly.

6. Consistency Analysis:

The system should compare:

- Feature model constraints (from Part 1)
- Code-derived feature dependencies

The tool should identify mismatches such as:

- Missing constraints
- Incorrect constraints
- Hidden dependencies

7. Inconsistency Detection (Mandatory Requirement):

The system must identify at least one inconsistency between:

- Feature model
- Code dependencies

The inconsistency must be:

- Clearly highlighted
- Classified (missing, incorrect, or hidden constraint)

8. Evidence and Justification:

For each identified dependency or inconsistency, the system must provide evidence such as:

- Code snippet
- Import/include statement
- File reference

Example:

```
# payment.py  
import auth
```

Unsupported claims will receive no credit

9. Impact Analysis:

The tool should analyze the impact of feature selection on the codebase.

When a user selects features, the system should display:

- Associated files/modules
- Indirect dependencies triggered

10. Explanation and Reporting:

The system (or report) should provide explanations for identified inconsistencies, including:

- Why the inconsistency occurs
- Which assumption in the feature model is incorrect
- How the issue can be resolved

Important Notes:

You are NOT required to build a full static analysis tool.

You are NOT required to perform deep code parsing (e.g., AST-level analysis).

A semi-manual approach with correct reasoning is sufficient.

Minimum Requirements Summary:

- Codebase size: 10–25 files
- Feature-to-code mappings: at least 5
- Dependencies identified: at least 5
- Inconsistencies detected: at least 1
- Evidence: mandatory for all claims

Technologies to Use:

- Any of your choice.
- **SAT Solver (suggestions):** PySAT (Python), Sat4j (Java) for logical validation.

Deliverables:

1. **Software Application:** A complete, functional tool for XML input, logic translation, MWP identification, and model visualization.
2. **User Guide:** Instructions on using the software.
3. **Demo Video:** (maximum 5 minutes with voice-over) A walkthrough demonstrating key features of the tool.

Evaluation Criteria:

Component	Marks	CLO
XML Parsing & Validation	10	CLO1
Logic Translation	20	CLO1
SAT Verification	10	CLO2
MWP Calculation	15	CLO2
Visualization & UI	20	CLO2
Constraint Handling	15	CLO2
Feature–Code Mapping	5	CLO2
Dependency Extraction	5	CLO2
Analysis Quality	10	CLO3
Inconsistency Detection	10	CLO3
User Guide	15	CLO3
Demo Video	15	CLO3
Total	150	

Best of Luck ☺